

`super()` to inherit properly from `Sprite`. We also set attributes for the screen and settings objects, and for the bullet's color.

Next we create the bullet's `rect` attribute ❶. The bullet isn't based on an image, so we have to build a `rect` from scratch using the `pygame.Rect()` class. This class requires the *x*- and *y*-coordinates of the top-left corner of the `rect`, and the width and height of the `rect`. We initialize the `rect` at (0, 0), but we'll move it to the correct location in the next line, because the bullet's position depends on the ship's position. We get the width and height of the bullet from the values stored in `self.settings`.

We set the bullet's `midtop` attribute to match the ship's `midtop` attribute ❷. This will make the bullet emerge from the top of the ship, making it look like the bullet is fired from the ship. We use a float for the bullet's *y*-coordinate so we can make fine adjustments to the bullet's speed ❸.

Here's the second part of *bullet.py*, `update()` and `draw_bullet()`:

```
bullet.py    def update(self):
              """Move the bullet up the screen."""
              # Update the exact position of the bullet.
❶            self.y -= self.settings.bullet_speed
              # Update the rect position.
❷            self.rect.y = self.y

              def draw_bullet(self):
                  """Draw the bullet to the screen."""
❸            pygame.draw.rect(self.screen, self.color, self.rect)
```

The `update()` method manages the bullet's position. When a bullet is fired, it moves up the screen, which corresponds to a decreasing *y*-coordinate value. To update the position, we subtract the amount stored in `settings.bullet_speed` from `self.y` ❶. We then use the value of `self.y` to set the value of `self.rect.y` ❷.

The `bullet_speed` setting allows us to increase the speed of the bullets as the game progresses or as needed to refine the game's behavior. Once a bullet is fired, we never change the value of its *x*-coordinate, so it will travel vertically in a straight line even if the ship moves.

When we want to draw a bullet, we call `draw_bullet()`. The `draw.rect()` function fills the part of the screen defined by the bullet's `rect` with the color stored in `self.color` ❸.

Storing Bullets in a Group

Now that we have a `Bullet` class and the necessary settings defined, we can write code to fire a bullet each time the player presses the spacebar. We'll create a group in `AlienInvasion` to store all the active bullets so we can manage the bullets that have already been fired. This group will be an instance of the `pygame.sprite.Group` class, which behaves like a list with some extra functionality that's helpful when building games. We'll use this group to draw bullets to the screen on each pass through the main loop and to update each bullet's position.